Python 编程 · 第 3–4 章 作业

作业逐题解析

字符串 · 列表 · 元组 · 字典

活动 3.1/4.1 · 第3章练习1–3 + 附加 · 第4章练习1–2 + 附加

2026年10月29日

目录

第0部分　作业总览

第1部分　活动 3.1 / 活动 4.1：打印第 5 个索引，切片索引 4 到 8

第2部分　第3章 练习1：把第 5 个索引换成 "P"，末尾接上 "world"

第3部分　第3章 练习2：长度、转大写、最大字符

第4部分　第3章 练习3：对齐、rpartition、数行数

第5部分　第3章 附加练习：重复 3 次；用 format() 截取

第6部分　第4章 练习1：列表操作

第7部分　第4章 练习2：元组操作

第8部分　第4章 附加练习：字典添加、删除键

第9部分　交作业前的检查清单

所有答案代码均在 Python 3.13 中实际运行；课本示例里的错误已标注并改正。

第0部分　作业总览

这份作业一共 **8 道题**，第3章考字符串，第4章考列表、元组、字典。每道题都按“**题目 → 思路 → 代码 → 运行结果 → 讲解**”来写。代码全部在 Python 3.13 里真实运行过；需要键盘输入的题，模拟敲入的内容写在运行结果的标签上。

题目	考点	用到的知识	代码文件
活动 3.1（= 活动 4.1）	字符串索引、切片	复习资料第8部分	`activity_3_1.py`
第3章 练习1	字符串不可变、拼接	复习资料第9、10部分	`ch3_ex1.py`
第3章 练习2	`len` 、 `upper` 、 `max`	复习资料第11部分	`ch3_ex2.py`
第3章 练习3	对齐函数、 `rpartition` 、数行数	本资料补充讲解	`ch3_ex3.py`
第3章 附加练习	重复 `*` 、 `format` 截取	复习资料第10、11部分	`ch3_extra.py`
第4章 练习1	列表 `append` `insert` `index`	第4章学习包 2.5、2.7	`ch4_ex1.py`
第4章 练习2	元组索引、切片、长度	第4章学习包第3部分	`ch4_ex2.py`
第4章 附加练习	字典增加、删除键	第4章学习包 5.3、5.4	`ch4_extra.py`

> **建议的做法：** 先看完《第1-3章复习》的索引切片部分，再来做作业。每道题**先自己写**，写不出来或运行报错再看这里的答案。8 个 `.py` 文件也一起发给你了，可以直接放进 PyCharm 里对照运行。

0.1 在 PyCharm 里建项目和文件

题目的“注意”要求：第3章的题放在项目 **StringPythonDemo** 里，第4章的题放在项目 **FirstPythonDemo** 里。

1. 打开 PyCharm， `File → New Project` ，项目名填 `StringPythonDemo` （或 `FirstPythonDemo` ），点 Create。
2. 在左边项目名上右键 → `New → Python File` ，输入文件名（如 `Test` ，PyCharm 会自动加 `.py` ）。
3. 把代码写进去，在代码区右键 → `Run 'Test'` 。
4. 需要输入的题，程序会停在下方 Run 窗口等你，**点一下 Run 窗口再打字**，按回车。

0.2 先说清楚：作业里有几处“坑”

做题前要知道，课本这几页里有几个地方写得不严谨，我都实际运行验证过：

位置	问题	这份解析的处理
活动 3.1 代码	`str = input() print(...)` 挤在一行，是排版问题，原样敲进去会报语法错误	拆成两行
活动 3.1 代码	用 `str` 做变量名，会把内置函数 `str()` 挡住	改名为 `s`
活动 3.1 输出	课本写切片结果是 `o Wo`，实际运行 `str[4:9]` 得到的是 `o Wor`	以真实运行结果为准
活动 3.1 题目	题目说“第 5 个字符”，代码打印的是“第 5 个索引”（其实是第 6 个字符）	按代码的“第 5 个索引”做，并解释区别
活动 4.1	和活动 3.1 一字不差，应该是课本复制重复了	只讲一次
第4章 练习 1	“安”应该是英文原书人名 Ann 翻译过来的	按题目原文用“安”，换成 `"Ann"` 也对
第4章 练习 2	“第三个索引”是 `coral[3]`，“第三个元素”才是 `coral[2]`，两者不同	按字面用 `coral[3]`，并说明区别

如果老师上课讲的要求和这里不一样，**以老师为准**。

第1部分　活动 3.1 / 活动 4.1：打印第 5 个索引，切片索引 4 到 8

题目

编写程序：打印输入字符串的第 5 个（索引的）字符，并把字符串从第 4 个索引切片到第 8 个索引（**包括两个索引**）。

思路

- 第 5 个索引的字符：`s[5]`
- 从索引 4 到索引 8 **都要包括**：切片含头不含尾，结束位置要写 **8 + 1 = 9**，所以是 `s[4:9]`

代码（改正后）

代码

```
# 活动 3.1 / 活动 4.1：打印第 5 个索引的字符，并切出索引 4 到 8（包括两端）
print('Enter the String')
s = input()
print('Input String =', s)
print('5th Index character =', s[5])
print('slice of String from 4th index to 8th index =', s[4:9])
```

运行结果　·　模拟键盘输入：Hello World

```
Enter the String
Hello World
Input String = Hello World
5th Index character =
slice of String from 4th index to 8th index = o Wor
```

逐个索引看清楚

字符	H	e	l	l	o	(空格)	W	o	r	l	d
索引	0	1	2	3	4	5	6	7	8	9	10

- `s[5]` 是**空格**，所以“5th Index character =”后面看起来什么都没有——不是程序错了，是那里真的是个空格。
- `s[4:9]` 取索引 4、5、6、7、8 → `o`、空格、`W`、`o`、`r` → **`o Wor`**。课本上印的 `o Wo` 少了一个 `r`，那其实是 `s[4:8]` 的结果。

> **“第 5 个字符”和“第 5 个索引”不一样：** 第 5 个字符是从 1 数的，对应索引 4，是 `o`；第 5 个索引是 `s[5]`，是空格。课本代码打印的是后者。考试时看清楚题目问的是哪个。

第2部分　第3章 练习1：把第 5 个索引换成 "P"，末尾接上 "world"

题目

对输入的字符串：1. 用字母 "P" 更新第 5 个索引；2. 在字符串末尾连接单词 "world"。

思路

字符串**不可变**，不能写 `s[5] = "P"`（会报 TypeError）。办法是**拆开再拼**：

> **新字符串 = 索引 5 之前的部分 + "P" + 索引 5 之后的部分**
>
> 也就是 `s[:5] + "P" + s[6:]`

- `s[:5]`：索引 0~4（不含 5）
- `"P"`：放在索引 5 的位置
- `s[6:]`：从索引 6 一直到最后，跳过了原来的索引 5

代码

代码

```
# 第3章 练习1：用字母 "P" 更新第 5 个索引；在末尾连接 "world"
s = input("请输入一个字符串：")

# 1. 字符串不可变，不能写 s[5] = "P"，只能拼一个新字符串
new_s = s[:5] + "P" + s[6:]
print("更新第5个索引后：", new_s)

# 2. 在末尾连接 "world"
result = new_s + "world"
print("末尾连接 world 后：", result)
```

运行结果　·　模拟键盘输入：Hello World

```
请输入一个字符串：Hello World
更新第5个索引后： HelloPWorld
末尾连接 world 后： HelloPWorldworld
```

讲解

输入 `Hello World` 时，索引 5 原本是空格，被换成了 `P`，于是变成 `HelloPWorld`；再接上 `world` 得到 `HelloPWorldworld`。

> 如果你想让“world”前面有个空格，就写 `new_s + " world"`。题目没要求，所以这里直接连接。

第3部分　第3章 练习2：长度、转大写、最大字符

题目

对输入的字符串：1. 打印长度；2. 转换成大写并打印；3. 打印字符串中最大的字母字符。

思路

三个都是现成的工具：`len(s)`、`s.upper()`、`max(s)`。

代码

代码

```
# 第3章 练习2：长度、转大写、最大字母字符
s = input("请输入一个字符串：")

print("字符串长度：", len(s))       # 1. 长度
print("转成大写：", s.upper())       # 2. 大写
print("最大的字符：", max(s))        # 3. 按编码比较，最大的那个字符
```

运行结果　·　模拟键盘输入：Hello World

```
请输入一个字符串：Hello World
字符串长度： 11
转成大写： HELLO WORLD
最大的字符： r
```

讲解

- **长度 11**：`Hello World` 是 10 个字母 + 1 个空格，空格也算一个字符。
- **最大字符是 `r`**：`max` 按字符编码比大小，规则是 **空格 < 数字 < 大写字母 < 小写字母**，同类按字母表顺序。小写字母里 `r` 在字母表最靠后（比 d、e、l、o 都靠后），所以是 `r`。大写的 `W` 虽然在字母表里更靠后，但大写字母整体比小写字母“小”。

> **易错：** `upper()` 不会改变原来的 `s`（字符串不可变），它是**生成一个新的大写字符串**。所以如果后面还要用大写版本，要存起来：`big = s.upper()`。

第4部分　第3章 练习3：对齐、rpartition、数行数

这道题有三小问，前两问是“**用什么函数**”的问答，第三问是“**算一个数**”。

4.1 用字符串函数对 3 行文字进行对齐

Python 字符串有三个对齐方法，都是“**把字符串放进一个固定宽度的格子里，空余的地方用填充字符补齐**”：

方法	作用	写法
`ljust(宽度, 填充字符)`	左对齐	`"Python".ljust(20, "*")`
`center(宽度, 填充字符)`	居中	`"Python".center(20, "*")`
`rjust(宽度, 填充字符)`	右对齐	`"Python".rjust(20, "*")`

填充字符不写的话默认是空格。这里用 `*` 是为了让你看清楚补了多少位。

4.2 哪个函数在“最后一次出现”的位置分成三段?

> **答案：`rpartition()`**
>
> `s.rpartition(分隔符)` 从**右边**开始找分隔符，在它**最后一次出现**的地方切开，返回一个**三元组**：`(分隔符前面的部分, 分隔符, 分隔符后面的部分)`。

对比 `partition()`：它在分隔符**第一次出现**的地方切开。`r` 就是 right（从右边）的意思。

4.3 计算给定段落的行数

思路： 每一行结尾都有一个换行符 `\n`，最后一行没有。所以 **行数 = 换行符个数 + 1**。也可以用 `splitlines()` 把段落按行拆成列表，再用 `len()` 数有几个。

题目给的段落在课本上排成 4 行，用三重引号原样写进程序，就保留了这 4 行。

代码（三小问合在一起）

代码

```
# 第3章 练习3：对齐、rpartition、数行数

# 1. 用字符串函数把 3 行文字对齐（宽度 20）
line1 = "Python"
line2 = "is"
line3 = "fun"
print("--- 左对齐 ljust ---")
print(line1.ljust(20, "*"))
print(line2.ljust(20, "*"))
print(line3.ljust(20, "*"))
print("--- 居中 center ---")
print(line1.center(20, "*"))
print(line2.center(20, "*"))
print(line3.center(20, "*"))
print("--- 右对齐 rjust ---")
print(line1.rjust(20, "*"))
print(line2.rjust(20, "*"))
print(line3.rjust(20, "*"))

# 2. 在分隔符"最后一次"出现的位置分成三段 → rpartition()
text = "a-b-c-d"
print(text.rpartition("-"))   # 从最后一个 - 切开
print(text.partition("-"))    # 对比：partition 从第一个 - 切开

# 3. 计算段落的行数（按题目里排版的样子分成 4 行）
paragraph = """条件编程或分支是你每天、每时每刻都在做的事情。这是关于评估条件的，如果绿灯亮，我就可
以过马路，如果下雨，我就带上雨伞，如果上班迟到，我就给经理打电话。主要工具是 if 语
句，它有不同的形式和颜色，但它基本上是计算一个表达式，并根据结果选择要执行的代码部
分。像往常一样，让我们看一个例子。"""
print("行数（count 换行符 + 1）：", paragraph.count("\n") + 1)
print("行数（splitlines）：", len(paragraph.splitlines()))
```

运行结果

```
--- 左对齐 ljust ---
Python**************
is******************
fun*****************
--- 居中 center ---
*******Python*******
*********is*********
********fun*********
--- 右对齐 rjust ---
**************Python
******************is
*****************fun
('a-b-c', '-', 'd')
('a', '-', 'b-c-d')
行数（count 换行符 + 1）： 4
行数（splitlines）： 4
```

讲解

- **对齐**：宽度 20 的格子里，`Python` 占 6 位，剩下 14 位用 `*` 补；`center` 把补的位置分到两边。
- **rpartition**：`"a-b-c-d"` 里最后一个 `-` 在 c 和 d 之间，所以切成 `('a-b-c', '-', 'd')`；`partition` 在第一个 `-` 切，得到 `('a', '-', 'b-c-d')`。
- **行数是 4**：段落里有 3 个换行符，3 + 1 = 4。两种方法结果一样。

注意： 行数取决于你写进程序时怎么分行。如果把整段写成一行（中间没有换行），结果就是 1。这里按课本的排版分成了 4 行。

第5部分　第3章 附加练习：重复 3 次；用 format() 截取

题目

对输入的字符串：1. 重复 3 次并打印；2. 使用 `format()` 函数将字符串从第二个索引截断到第六个索引。

思路

1. 重复：`s * 3`
2. `format` 里的 `{:.n}` 用在字符串上，意思是“**最多只保留前 n 个字符**”。所以分两步：
3. 先用 `s[2:]` 从索引 2 开始取
4. 再用 `{:.4}` 只保留 4 个字符 → 正好是索引 2、3、4、5

代码

代码

```
# 第3章 附加练习: 重复 3 次; 用 format() 截取索引 2 到 6
s = input("请输入一个字符串: ")

# 1. 重复 3 次
print("重复3次: ", s * 3)

# 2. format 里 {:.n} 的意思是"最多只保留 n 个字符"
#    先从索引 2 开始取 s[2:], 再只保留 4 个字符 → 索引 2、3、4、5 (不含 6, 和切片规则一致)
print("索引2到6 (不含6): ", "{:.4}".format(s[2:]))
#    如果老师要求"包括第 6 个索引", 就保留 5 个字符 → 索引 2~6
print("索引2到6 (含6): ", "{:.5}".format(s[2:]))
```

运行结果 · 模拟键盘输入：Programming

```
请输入一个字符串: Programming
重复3次:  ProgrammingProgrammingProgramming
索引2到6 (不含6):  ogra
索引2到6 (含6):  ogram
```

讲解

`Programming`：P0 r1 o2 g3 r4 a5 m6 m7 i8 n9 g10。

- 索引 2 到 6、**不含 6**（和切片 `[2:6]` 一样）：o g r a → `ogra`
- 如果老师要求**包括第 6 个索引**：o g r a m → `ogram`

题目没写“包括”，所以第一种是按 Python 切片习惯的答案；第二种是备选，两种都写在代码里了，交作业时按老师的要求留一种就行。

> 不用 `format` 的话，`s[2:6]` 一步就能得到同样结果。题目特意要求用 `format`，就是考你知道 `{:.n}` 能截断字符串。

第6部分　第4章 练习1：列表操作

题目

列表 `['John', 'Allen', 'Black', 'Raj', 'Krish', 'Peter', 'Mike']`：1. 在末尾添加“安”；2. 在第三个索引处插入 “Ricky”；3. 打印单词 “Peter” 的索引。

思路

要求	用什么
末尾添加	`append("安")`
在索引 3 插入	`insert(3, "Ricky")`
查索引	`index("Peter")`

代码

代码

```
# 第4章 练习1：列表操作
names = ['John', 'Allen', 'Black', 'Raj', 'Krish', 'Peter', 'Mike']
print("原列表：", names)

# 1. 在列表末尾添加 "安"
names.append("安")
print("append 后：", names)

# 2. 在索引 3 的位置插入 "Ricky"
names.insert(3, "Ricky")
print("insert 后：", names)

# 3. 打印 "Peter" 的索引
print("Peter 的索引：", names.index("Peter"))
```

运行结果

```
原列表： ['John', 'Allen', 'Black', 'Raj', 'Krish', 'Peter', 'Mike']
append 后： ['John', 'Allen', 'Black', 'Raj', 'Krish', 'Peter', 'Mike', '安']
insert 后： ['John', 'Allen', 'Black', 'Ricky', 'Raj', 'Krish', 'Peter', 'Mike', '安']
Peter 的索引： 6
```

讲解

- `append` 把“安”放到最后。
- `insert(3, "Ricky")` 让 Ricky 站到索引 3，原来索引 3 的 Raj 和后面的人**都往后挪一位**。
- 所以 Peter 从原来的索引 **5** 变成了 **6**。

> **易错：操作顺序会影响结果。** 如果先查 Peter 的索引再插入，得到的是 5。题目是按 1、2、3 的顺序做的，所以答案是 6。

第7部分　第4章 练习2：元组操作

题目

元组 `coral = ('blue coral', 'staghorn coral', 'pillar coral', 'elkhorn coral')`：1. 打印第三个索引值；2. 打印切片 `[1:3]`；3. 打印元组长度。

代码

代码

```
# 第4章 练习2：元组操作
coral = ('blue coral', 'staghorn coral', 'pillar coral', 'elkhorn coral')

# 1. 打印"第三个索引"的值，也就是 coral[3]
print("coral[3] =", coral[3])

# 2. 打印切片 [1:3]：索引 1 和 2（不含 3）
print("coral[1:3] =", coral[1:3])

# 3. 打印元组长度
print("长度 =", len(coral))
```

运行结果

```
coral[3] = elkhorn coral
coral[1:3] = ('staghorn coral', 'pillar coral')
长度 = 4
```

讲解

元素	'blue coral'	'staghorn coral'	'pillar coral'	'elkhorn coral'
索引	0	1	2	3

- **第三个索引** `coral[3]` → `elkhorn coral`
- **切片** `[1:3]` → 索引 1、2（不含 3）→ `('staghorn coral', 'pillar coral')`，结果仍然是**元组**（带圆括号）
- **长度** `len(coral)` → 4

> **“第三个索引” ≠ “第三个元素”：** 第三个元素从 1 数，是 `coral[2]`，即 `pillar coral`。题目写的是“第三个索引”，所以用 `coral[3]`。如果老师的意思是第三个元素，就改成 `coral[2]`。

第8部分　第4章 附加练习：字典添加、删除键

题目

字典 `sammy = {'username': 'sammy-shark', 'online': True, 'followers': 987}`：1. 添加一个键，打印新字典；2. 删除 `"online"` 键值对，打印新字典。

思路

- 添加：`字典[新键] = 值`（键不存在就是新增）
- 删除：`del 字典[键]`，或者 `字典.pop(键)`

代码

代码

```
# 第4章 附加练习：字典添加键、删除键
sammy = {'username': 'sammy-shark', 'online': True, 'followers': 987}

# 1. 添加一个新键，再打印
sammy['location'] = 'Beijing'
print("添加键后：", sammy)

# 2. 删除 "online" 键值对，再打印
del sammy['online']
print("删除 online 后：", sammy)
```

运行结果

```
添加键后： {'username': 'sammy-shark', 'online': True, 'followers': 987, 'location': 'Beijing'}
删除 online 后： {'username': 'sammy-shark', 'followers': 987, 'location': 'Beijing'}
```

讲解

题目没规定加什么键，我加的是 `'location': 'Beijing'`，你可以换成任何键，比如 `sammy['age'] = 18`。删除用 `del`，用 `sammy.pop('online')` 也完全正确，区别是 `pop` 会把被删的值（`True`）交回来。

第9部分　交作业前的检查清单

- 每个文件都能运行，没有红色报错
- 第3章的文件在 `StringPythonDemo` 项目里，第4章的在 `FirstPythonDemo` 项目里
- 活动 3.1 已经把挤在一行的代码拆开，变量名不用 `str`
- 理解每道题的“为什么”，老师抽问时能讲出来：
- 为什么 `s[4:9]` 而不是 `s[4:8]`？（含头不含尾，要包括 8 就写 9）
- 为什么不能 `s[5] = "P"`？（字符串不可变）
- 为什么 Peter 的索引是 6？（insert 让后面的元素后移）
- `rpartition` 和 `partition` 有什么区别？（最后一次 vs 第一次）